

EA 3120 - Data Acquisition for IoT

Practical 2

Name : Lilan Manoj
Index No: 24SEA052

Exercise 01

1. The Frequency

Frequency is the number of cycles of a waveform completed per second. Determines how fast a signal oscillates.

Unit: Hertz (Hz)

$$\text{Frequency } (f) = \frac{1}{T \text{ (Time Period)}}$$

2. Time Period

Time period is the time taken to complete one full cycle of a waveform.

Unit: Seconds (s)

$$\text{Time Period } (T) = \frac{1}{f \text{ (Frequency)}}$$

3. Amplitude

Amplitude is the maximum value of a signal measured from its zero or mean level. Which indicates signal strength.

4. Peak to peak value

Peak-to-peak value is the difference between the maximum positive and maximum negative values of a waveform.

$$V_{pp} = V_{max} - V_{min}$$

5. Resolution of ADC or DAC

Resolution is the smallest change in analog value that can be detected or produced.

$$\text{Resolution} = \frac{\text{Full Scale Range}}{2^n}; \text{ Where } n = \text{number of bits.}$$

6. Sample Rate (Sampling Frequency)

Sample rate is the number of samples taken per second from an analog signal.

Unit: Samples/second or Hz

7. Bit Rate

Bit rate is the number of bits transmitted per second.

Unit: bits per second (bps)

$$\text{Bit rate} = n \times f_s$$

Where:

n = number of bits.

f_s = Sample rate

8. Offset

Offset is the DC component or shift of a signal from zero level. Causes waveform to shift upward or downward.

9. Phase Shift

Phase shift is the angular displacement of a waveform relative to a reference signal.

Unit: Degrees or radians

$$\phi = \omega t$$

10. SNR (Signal to Noise Ratio)

SNR is the ratio of signal power to noise power.

$$SNR = \frac{P_s}{P_N}$$

In dB (Decibels):

$$SNR_{dB} = 10 \log_{10} \left(\frac{P_s}{P_N} \right)$$

Where:

Ps = Power of signal

Pn = Power of noise

11. Duty Cycle

Duty cycle is the percentage of time a signal remains ON during one period. Used mainly in digital and pulse signals.

$$\text{Duty cycle} = \frac{T_{ON}}{T} \times 100\%$$

12. Falling Time, Rising Time

Indicates speed of signal transitions.

Rise Time:

Time taken for a signal to rise from **10% to 90%** of its maximum value.

Fall Time:

Time taken for a signal to fall from **90% to 10%** of its maximum value.

13. Nyquist Theorem / Nyquist Frequency

Nyquist Theorem:

A signal can be reconstructed if, $f_s \geq 2f_{max}$

Nyquist Frequency:

$$\text{Nyquist Frequency } (f_N) = \frac{f_s}{2}$$

Where:

fs = Sampling Frequency

fmax = Maximum Frequency Component

fN = Nyquist Frequency

14. Signal Aliasing

Aliasing is the distortion that occurs when a signal is sampled below the Nyquist rate (under sampled), causing high-frequency components to appear as low-frequency components.

15. Quantization Error

Quantization error is the difference between the actual analog value and its quantized digital value. Can be reduced by increasing the number of bits.

$$eq = x(t) - xq(t)$$

Exercise 02

Code:

```
#include "driver/dac.h"

#define DAC_PIN 25

const unsigned long stepInterval = 1;

int dacValue = 0;
int direction = 1;
unsigned long previousMillis = 0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  unsigned long currentMillis = millis();

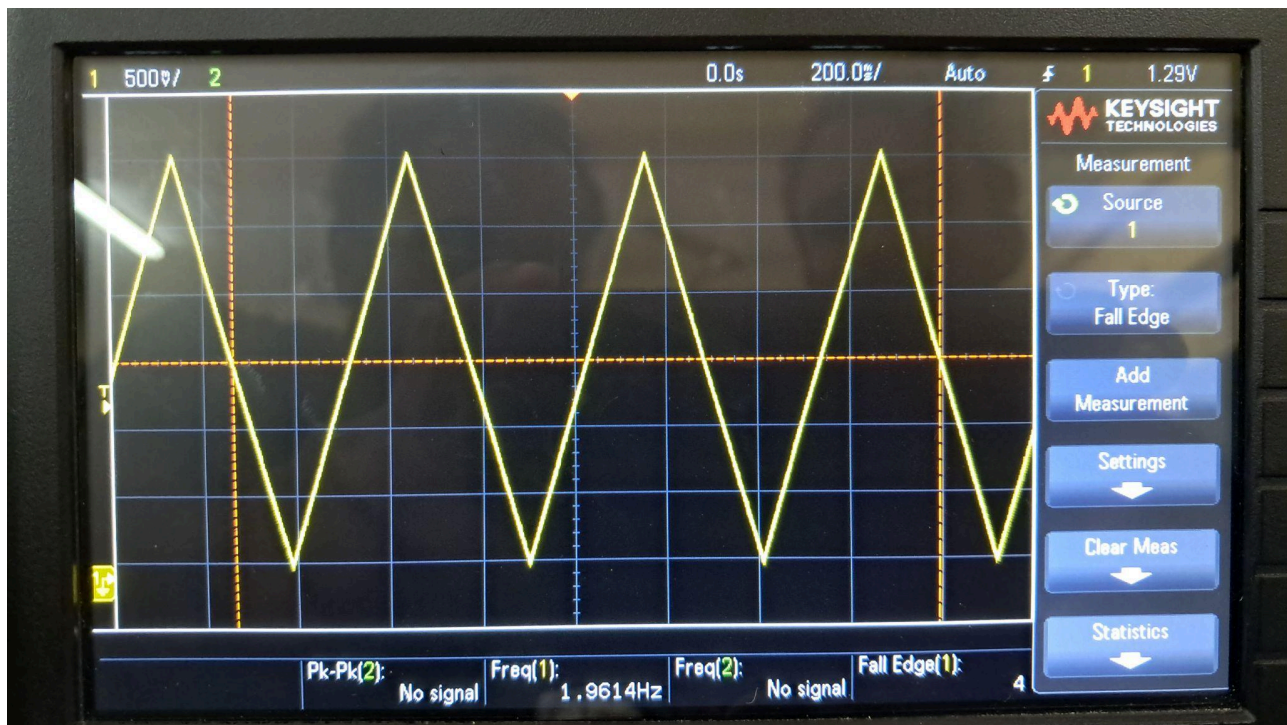
  if (currentMillis - previousMillis >= stepInterval) {
    previousMillis = currentMillis;

    dacWrite(DAC_PIN, dacValue);

    dacValue += direction;

    if (dacValue >= 255) {
      dacValue = 255;
      direction = -1;
    } else if (dacValue <= 0) {
      dacValue = 0;
      direction = 1;
    }
  }
}
```

Waveform/Results



- a.) Time period (T) = 0.509 s
- b.) Expected waveform: Sawtooth Wave
- c.) Frequency (f) = 1.961 Hz
- d.) Expected offset = 0s
- e.) peak to peak value = 3.43 V
- f.) Sample rate = 1000 sample/s
- g.) Resolution = 0.013 V
- i.) Frequency of the waveform can be changed, if we decrease/increase the step interval.

Exercise 03

Code:

```
#include "soc/rtc_io_reg.h"
#include "soc/rtc_cntl_reg.h"
#include "soc/sens_reg.h"
#include "soc/rtc.h"
#include "driver/dac.h"

void setup() {
    // Setup Serial terminal
    Serial.begin(115200);
    delay(1000);
    SET_PERI_REG_MASK(SENS_SAR_DAC_CTRL1_REG, SENS_SW_TONE_EN);
    SET_PERI_REG_MASK(SENS_SAR_DAC_CTRL2_REG, SENS_DAC_CW_EN1_M);
    SET_PERI_REG_BITS(SENS_SAR_DAC_CTRL2_REG, SENS_DAC_INV1, 2,
    SENS_DAC_INV1_S);
```

```

// Invert MSB
SET_PERI_REG_BITS(SENS_SAR_DAC_CTRL2_REG, SENS_DAC_SCALE1, 0,
SENS_DAC_SCALE1_S); // set 0 -> 100% Amplitude
SET_PERI_REG_BITS(SENS_SAR_DAC_CTRL2_REG, SENS_DAC_DC1, 0, SENS_DAC_DC1_S);
// set offset 0 -> mid point
// the frequency into 1000Hz
long freqHz = 1000;
int result = 0;
double f, fTarget, delta, delta_min=99999.0;
int stepTarget=0, divTarget=0;

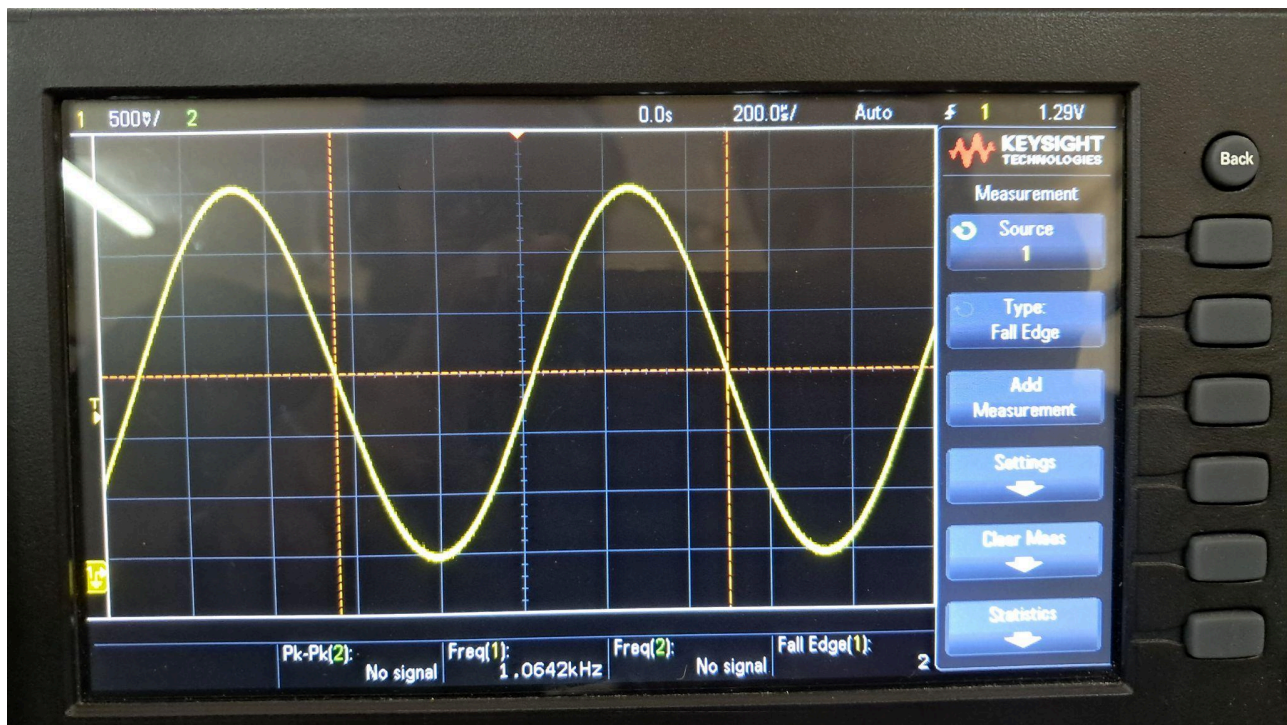
for (int step=1; step<2000; step++) {
    for (int divi=0; divi<8; divi++) {
        f = 125.6*(double)step/(double)(divi+1);
        delta = abs((freqHz - f));
        if(delta < delta_min){
            delta_min = delta;
            stepTarget = step;
            divTarget = divi;
            fTarget = f;
        }
    }
}

REG_SET_FIELD(RTC_CNTL_CLK_CONF_REG, RTC_CNTL_CK8M_DIV_SEL, divTarget);
SET_PERI_REG_BITS(SENS_SAR_DAC_CTRL1_REG, SENS_SW_FSTEP, stepTarget,
SENS_SW_FSTEP_S);
dac_output_enable(DAC_CHANNEL_1);
result = fTarget;
Serial.println("DAC Sin Started on GPIO25 ");
Serial.print(result); Serial.println(" Hz");
Serial.print(stepTarget); Serial.println(" Steps");
Serial.print(divTarget); Serial.println(" Div");
}

void loop() {
    // put your main code here, to run repeatedly:
}

```

Waveform/Results



Exercise 04

1. 0
2. 4095
3. $(3221 - 3224) \rightarrow 2.590$

Code:

```
// Potentiometer is connected to GPIO 1 (Analog ADC1_CH1)
#define POT_PIN 1
#define LED_PIN 42

// variable for storing the potentiometer value
int potValue = 0;
float voltage = 0;

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  delay(1000);
}

void loop() {
  // Reading potentiometer value
  potValue = analogRead(POT_PIN);
  voltage = (potValue * 3.3) / 4095.0;
```

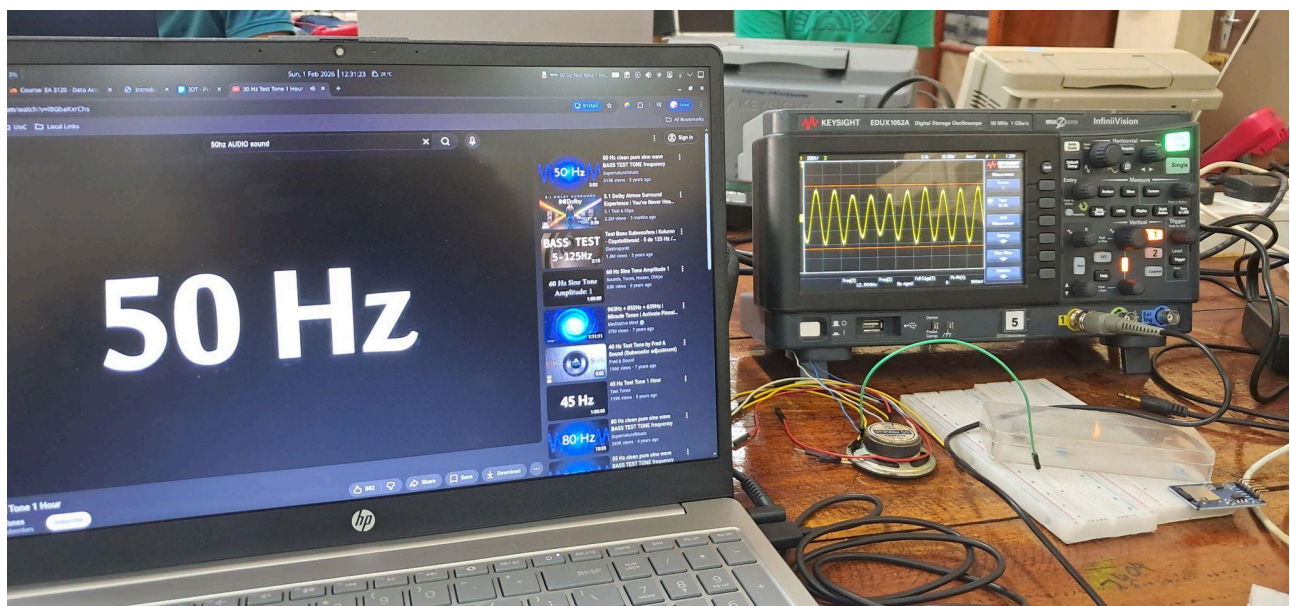
```
Serial.printf("%d \t %f\n", potValue, voltage);

if (voltage > 2.5) {
  digitalWrite(LED_PIN, HIGH);
} else {
  digitalWrite(LED_PIN, LOW);
}

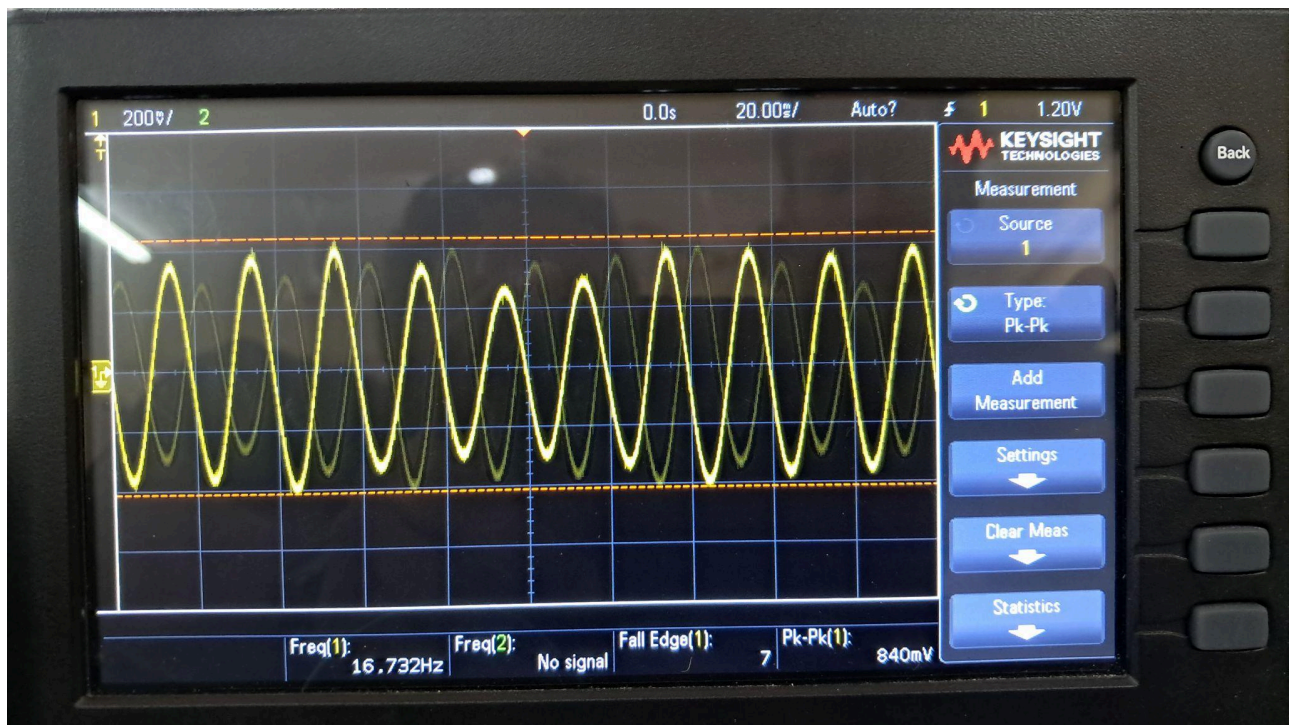
delay(500);
}
```

DSP Exercise

Setup:

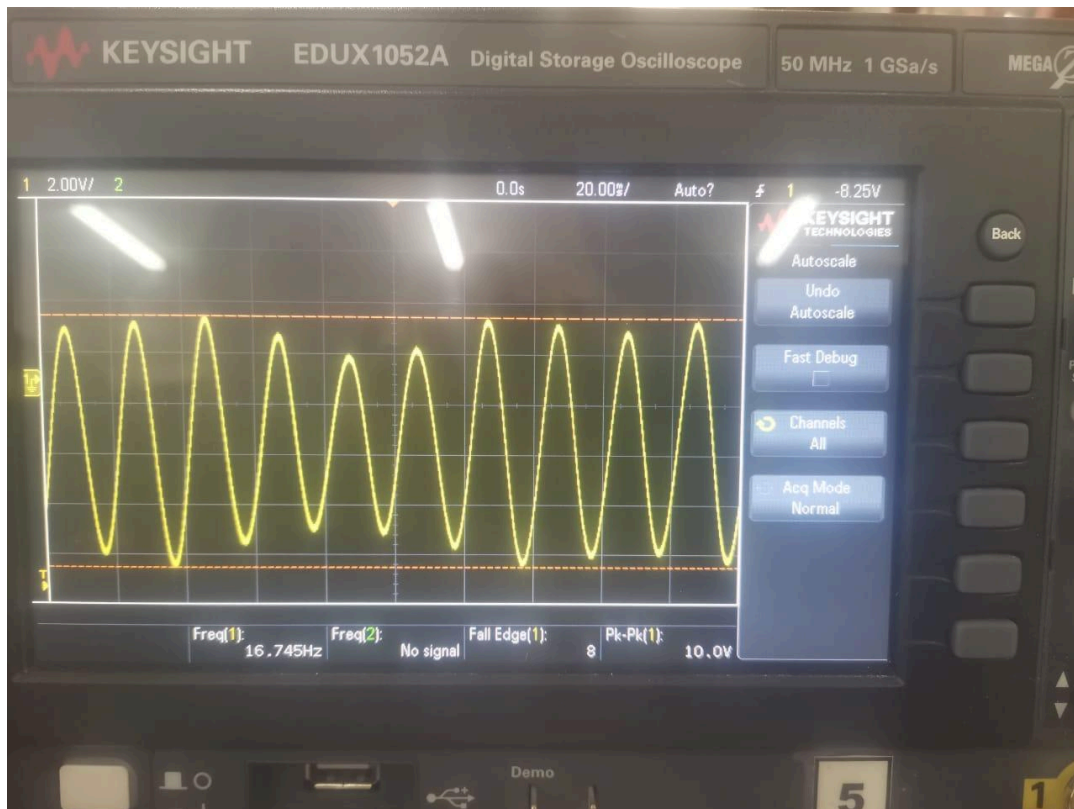


Acquired Signal Pre Amplification:



Pk-Pk = 840 mV

Post amplification:



Pk-Pk = 10 V